Report – Indexed File Organization

Introduction 1
Selected Indexing Method 1
Implementation Description 2
File Organization 2
Records 2
Tree Operations 3
Free Space Management 3
Buffering and Operation Counters 3
Test File Format 4
Presentation of Results 4
Description of Conducted Experiments 4
Methodology 5
Experiment Results Legend 5
Experimental Results 5
Experiment: Node Degrees 6
Experiment: Inverted Degrees 6
Experiment: Records 7
Analysis and Discussion of Results 8
Final Conclusions 9

Introduction

The objective of this task was to design and implement an indexed file organization that enables efficient storage and retrieval of records by key. The B+ tree was selected as the indexing method due to its high locality of reference, low height, and efficient handling of dynamic operations such as record insertion, deletion, and updates.

The implementation was developed in Python and includes a complete set of CRUD operations, a mechanism for reusing freed space, and a system for measuring the number of disk page read and write operations. Based on this, experiments were conducted to analyze the impact of implementation parameters on operational complexity.

Selected Indexing Method

A B+ tree was implemented with the following parameters:

d – internal node degree r – leaf node degree

Key characteristics of the applied structure:

- All records are stored exclusively in the leaves.
- Internal nodes contain only separator keys and pointers.
- Leaves are linked via a bidirectional list (prev, next), enabling efficient sequential data retrieval by keys.
- The tree is always balanced.

The implemented solution does not require manual file reorganization typically found in indexed-sequential structures. Logical reorganization is performed automatically through split and merge operations during record insertion and deletion.

Implementation Description

File Organization

The data file uses a paged structure and consists of:

- **File Header (FileHeader)**, containing:
    - page size,

    - root page identifier,
    - pointer to the free page list.
- **Data Pages (Page)**, which can be:
    - internal nodes (**INTERNAL**),
    - leaves (**LEAF**),
    - free pages (**FREE**).

Each page contains a **Page Header (PageHeader)** including:

- page type,
- page identifier,
- parent identifier,
- key count,
- **prev** and **next** pointers (for leaves),
- minimum and maximum key count.

Pages are binary serialized using the **struct** module.

Records

A record consists of:

- a natural number key,
- two float values.

Records are stored exclusively in the tree leaves.

Tree Operations

The following operations have been implemented:

- **Insert** – record insertion:
    - leaf node lookup,
    - key insertion,
    - in case of overflow: page **split** and upward propagation.
- **Search** – record retrieval:
    - traversal from root to leaf,
    - zero write operations.
- **Delete** – record removal:
    - key removal from the leaf,
    - in case of underflow: key borrowing from a sibling or page **merge**,
    - deallocated pages are added to the free page list.

- **Update** – record modification:
    - if the key is modified: executed as a **delete** followed by an **insert**,
    - otherwise: direct update of the record data within the leaf.

Free Space Management

- Deleted pages are marked as **FREE** and appended to a free page list.
- When creating new pages, the system prioritizes reusing pages from the free list. This limits file growth and fulfills the requirement for space reclamation.

Buffering and Operation Counters

A simple buffering mechanism was applied:

- The root page is always kept in the cache.
- The cache is cleared after each operation.
- A pessimistic assumption for the maximum number of pages in the cache was adopted:

$$3 \cdot (h - 1) + 1$$

where **h** represents the tree height.

Test File Format

The test file contains a sequence of tree commands in text format:

- insert <key> <float> <float>
- read <key>
- delete <key>
- update <old_key> <new_key> <float> <float>

The program supports:

- interactive mode,
- batch processing of commands from a test file.

Presentation of Results

After each operation that modifies the tree structure, the system can display:

- the B+ tree structure,

- key distribution within nodes,
- links between leaf nodes.

In the experiments, the following metrics are presented:

- average number of page reads and writes per operation,
- theoretical values (best case and worst case),
- total number of pages occupied by the file.

Description of Conducted Experiments

Methodology

The experiments were conducted according to the following procedure:

1. Populating the tree with a predefined number of records.
2. Creating a copy of the tree and the data file for each CRUD operation.
3. Executing a specified number of operations (insert, delete, search, update).
4. Calculating the average number of page read and write operations.

Experiments were carried out for:

- varying values of degrees **d** and **r** with a constant number of records,
- varying numbers of records with constant tree parameters.

Experiment Results Legend

Rows (Configuration):

- **(k, t)** – Structural tree parameters (**k**: max records per leaf, **t**: internal node degree).
- **Number of records** – Total number of elements inserted into the tree before measurements (data scale from 1,000 to 64,000).

Header: <Operation> <Scenario>-<I/O Type>

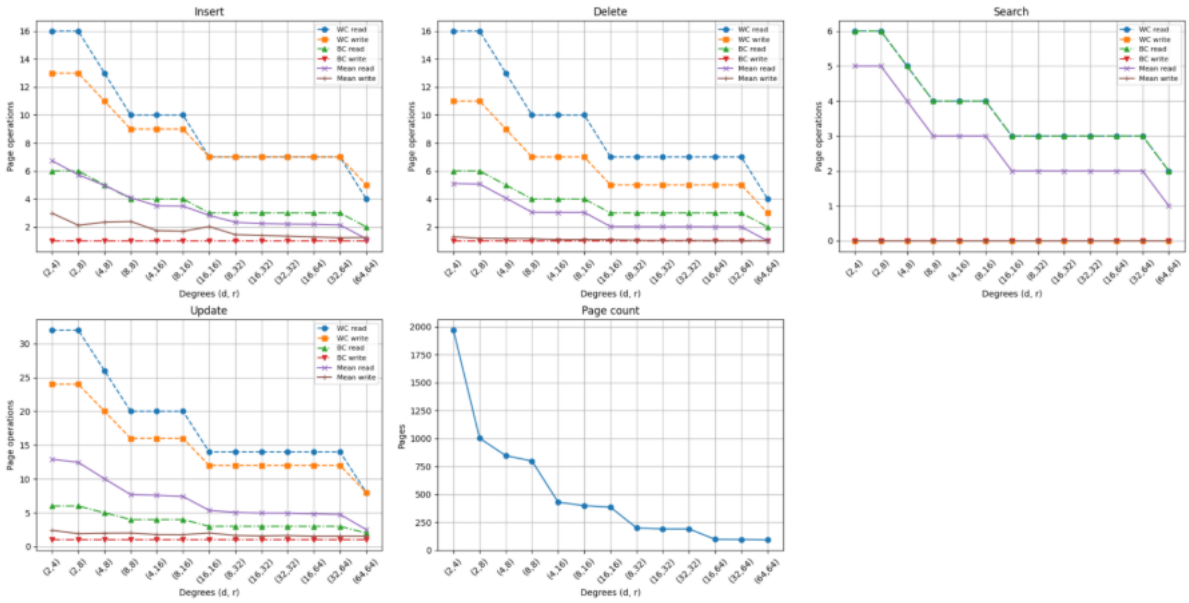
- **Operations:**
  - **I** (Insert), **D** (Delete), **S** (Search), **U** (Update).
- **Scenarios:**
  - **WC** (Worst Case) – e.g., an operation involving node splitting or merging.
  - **BC** (Best Case).
  - **μ** (Average Case) – Mean value from a series of tests.
- **I/O Type:**
  - **R** (Read) – Number of page reads from disk.
  - **W** (Write) – Number of page writes to disk.

Experimental Results

Experiment: Node Degrees

B+Tree Experiment Results

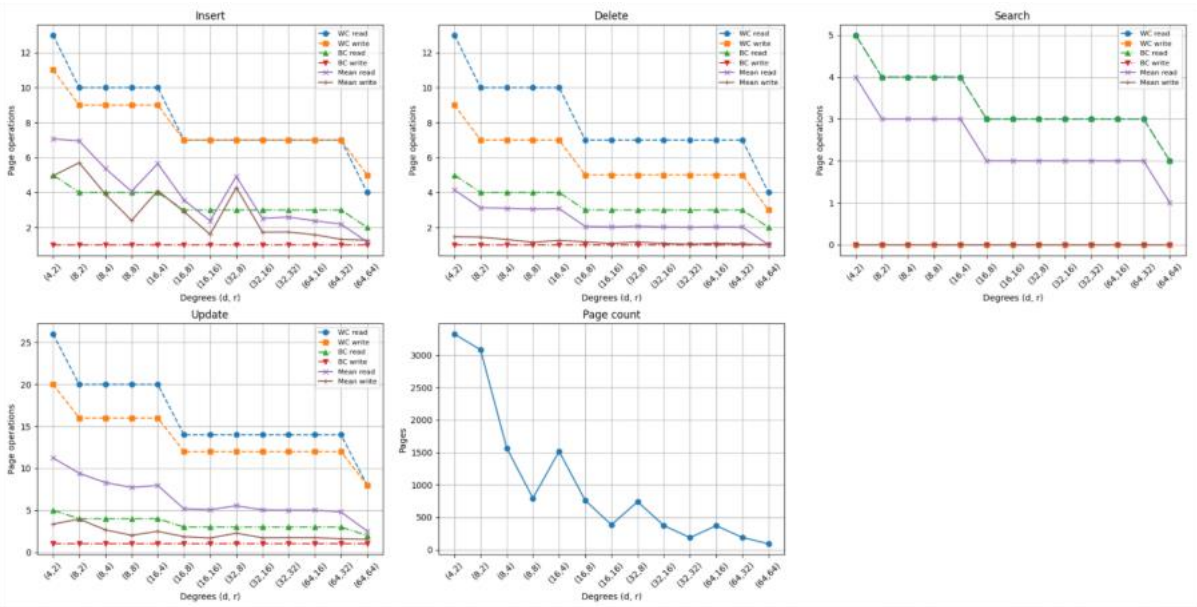
	I WC-R	I WC-W	I BC-R	I BC-W	I μ-R	I μ-W	D WC-R	D WC-W	D BC-R	D BC-W	D μ-R	D μ-W	S WC-R	S WC-W	S BC-R	S BC-W	S μ-R	S μ-W	U WC-R	U WC-W	U BC-R	U BC-W	U μ-R	U μ-W
1000	10.00	9.00	4.00	1.00	4.56	3.11	10.00	7.00	4.00	1.00	3.54	1.83	4.00	0.00	4.00	0.00	3.00	0.00	20.00	16.00	4.00	1.00	7.78	2.20
2000	10.00	9.00	4.00	1.00	4.57	3.08	10.00	7.00	4.00	1.00	3.18	1.42	4.00	0.00	4.00	0.00	3.00	0.00	20.00	16.00	4.00	1.00	7.91	2.32
4000	13.00	11.00	5.00	1.00	6.05	3.64	13.00	9.00	5.00	1.00	4.09	1.28	5.00	0.00	5.00	0.00	4.00	0.00	26.00	20.00	5.00	1.00	10.36	2.48
8000	13.00	11.00	5.00	1.00	5.70	3.23	13.00	9.00	5.00	1.00	4.10	1.31	5.00	0.00	5.00	0.00	4.00	0.00	26.00	20.00	5.00	1.00	10.43	2.48
16000	13.00	11.00	5.00	1.00	5.77	3.32	13.00	9.00	5.00	1.00	4.07	1.21	5.00	0.00	5.00	0.00	4.00	0.00	26.00	20.00	5.00	1.00	10.66	2.58
32000	16.00	13.00	6.00	1.00	6.65	3.20	16.00	11.00	6.00	1.00	5.06	1.25	6.00	0.00	6.00	0.00	5.00	0.00	32.00	24.00	6.00	1.00	12.99	2.63
64000	16.00	13.00	6.00	1.00	6.46	2.98	16.00	11.00	6.00	1.00	5.05	1.20	6.00	0.00	6.00	0.00	5.00	0.00	32.00	24.00	6.00	1.00	13.12	2.70



Experiment: Inverted Degrees

B+Tree Experiment Results

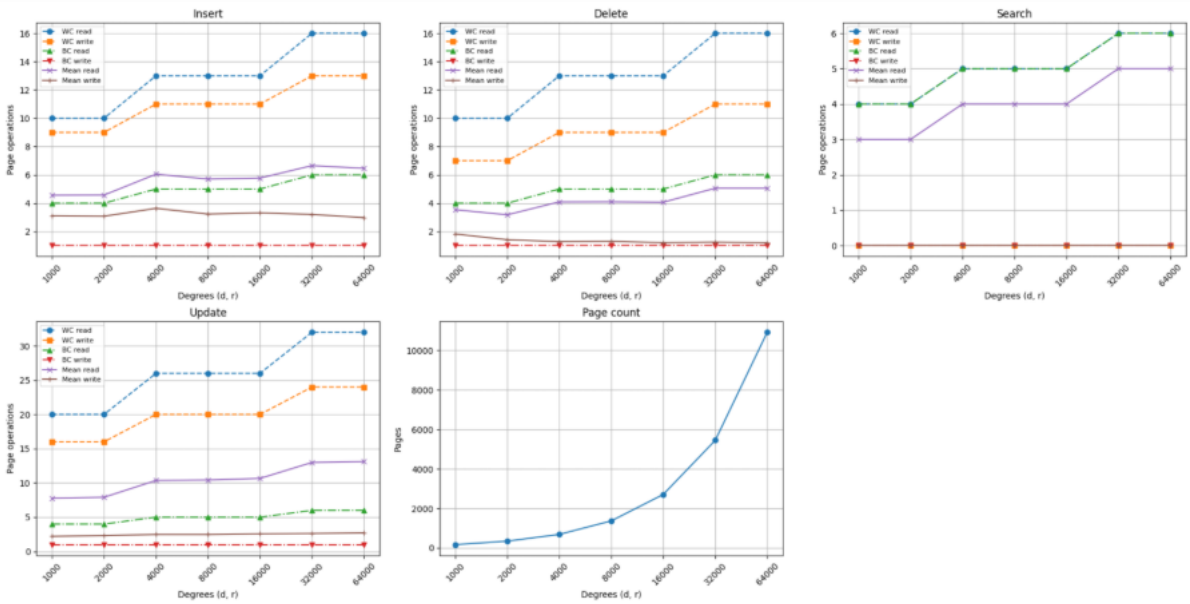
	I WC-R	I WC-W	I BC-R	I BC-W	I μ-R	I μ-W	D WC-R	D WC-W	D BC-R	D BC-W	D μ-R	D μ-W	S WC-R	S WC-W	S BC-R	S BC-W	S μ-R	S μ-W	U WC-R	U WC-W	U BC-R	U BC-W	U μ-R	U μ-W
(4,2)	13.00	11.00	5.00	1.00	7.07	4.95	13.00	9.00	5.00	1.00	4.15	1.46	5.00	0.00	5.00	0.00	4.00	0.00	26.00	20.00	5.00	1.00	11.24	3.36
(8,2)	10.00	9.00	4.00	1.00	6.95	5.70	10.00	7.00	4.00	1.00	3.12	1.45	4.00	0.00	4.00	0.00	3.00	0.00	20.00	16.00	4.00	1.00	9.41	3.95
(8,4)	10.00	9.00	4.00	1.00	5.37	3.90	10.00	7.00	4.00	1.00	3.10	1.31	4.00	0.00	4.00	0.00	3.00	0.00	20.00	16.00	4.00	1.00	8.30	2.66
(8,8)	10.00	9.00	4.00	1.00	4.06	2.39	10.00	7.00	4.00	1.00	3.05	1.15	4.00	0.00	4.00	0.00	3.00	0.00	20.00	16.00	4.00	1.00	7.73	2.02
(16,4)	10.00	9.00	4.00	1.00	5.65	4.10	10.00	7.00	4.00	1.00	3.08	1.26	4.00	0.00	4.00	0.00	3.00	0.00	20.00	16.00	4.00	1.00	7.96	2.49
(16,8)	7.00	7.00	3.00	1.00	3.56	2.90	7.00	5.00	3.00	1.00	2.05	1.17	3.00	0.00	3.00	0.00	2.00	0.00	14.00	12.00	3.00	1.00	5.18	1.86
(16,16)	7.00	7.00	3.00	1.00	2.38	1.60	7.00	5.00	3.00	1.00	2.03	1.08	3.00	0.00	3.00	0.00	2.00	0.00	14.00	12.00	3.00	1.00	5.07	1.69
(32,8)	7.00	7.00	3.00	1.00	4.91	4.28	7.00	5.00	3.00	1.00	2.06	1.17	3.00	0.00	3.00	0.00	2.00	0.00	14.00	12.00	3.00	1.00	5.54	2.28
(32,16)	7.00	7.00	3.00	1.00	2.52	1.73	7.00	5.00	3.00	1.00	2.03	1.08	3.00	0.00	3.00	0.00	2.00	0.00	14.00	12.00	3.00	1.00	5.05	1.72
(32,32)	7.00	7.00	3.00	1.00	2.59	1.74	7.00	5.00	3.00	1.00	2.01	1.04	3.00	0.00	3.00	0.00	2.00	0.00	14.00	12.00	3.00	1.00	5.01	1.73
(64,16)	7.00	7.00	3.00	1.00	2.37	1.58	7.00	5.00	3.00	1.00	2.03	1.09	3.00	0.00	3.00	0.00	2.00	0.00	14.00	12.00	3.00	1.00	5.02	1.73
(64,32)	7.00	7.00	3.00	1.00	2.19	1.32	7.00	5.00	3.00	1.00	2.02	1.05	3.00	0.00	3.00	0.00	2.00	0.00	14.00	12.00	3.00	1.00	4.80	1.61
(64,64)	4.00	5.00	2.00	1.00	1.17	1.26	4.00	3.00	2.00	1.00	1.00	1.02	2.00	0.00	2.00	0.00	1.00	0.00	8.00	8.00	2.00	1.00	2.51	1.53



Experiment: Records

B+Tree Experiment Results

	I WC-R	I WC-W	I BC-R	I BC-W	I μ-R	I μ-W	D WC-R	D WC-W	D BC-R	D BC-W	D μ-R	D μ-W	S WC-R	S WC-W	S BC-R	S BC-W	S μ-R	S μ-W	U WC-R	U WC-W	U BC-R	U BC-W	U μ-R	U μ-W
1000	10.00	9.00	4.00	1.00	4.56	3.11	10.00	7.00	4.00	1.00	3.54	1.83	4.00	0.00	4.00	0.00	3.00	0.00	20.00	16.00	4.00	1.00	7.78	2.20
2000	10.00	9.00	4.00	1.00	4.57	3.08	10.00	7.00	4.00	1.00	3.18	1.42	4.00	0.00	4.00	0.00	3.00	0.00	20.00	16.00	4.00	1.00	7.91	2.32
4000	13.00	11.00	5.00	1.00	6.05	3.64	13.00	9.00	5.00	1.00	4.09	1.28	5.00	0.00	5.00	0.00	4.00	0.00	26.00	20.00	5.00	1.00	10.36	2.48
8000	13.00	11.00	5.00	1.00	5.70	3.23	13.00	9.00	5.00	1.00	4.10	1.31	5.00	0.00	5.00	0.00	4.00	0.00	26.00	20.00	5.00	1.00	10.43	2.48
16000	13.00	11.00	5.00	1.00	5.77	3.32	13.00	9.00	5.00	1.00	4.07	1.21	5.00	0.00	5.00	0.00	4.00	0.00	26.00	20.00	5.00	1.00	10.66	2.58
32000	16.00	13.00	6.00	1.00	6.65	3.20	16.00	11.00	6.00	1.00	5.06	1.25	6.00	0.00	6.00	0.00	5.00	0.00	32.00	24.00	6.00	1.00	12.99	2.63
64000	16.00	13.00	6.00	1.00	6.46	2.98	16.00	11.00	6.00	1.00	5.05	1.20	6.00	0.00	6.00	0.00	5.00	0.00	32.00	24.00	6.00	1.00	13.12	2.70



Analysis and Discussion of Results

Based on the results, the following trends can be observed:

- **Increasing the degrees d and r leads to:**
    - reduced tree height,
    - a decrease in the number of read operations during search and update.
- **Significant disproportion in the case of d > r (top-heavy tree)** causes an increase in the number of reads and writes.
- **Search operations** are characterized by zero writes and a read count of **h - 1** due to root buffering.
- **Update operations** are the most costly, especially when a key is modified, as they combine both record deletion and insertion.
- **As the number of records increases**, the number of operations grows logarithmically, aligning with the theoretical expectations for B+ trees.

Configurations closest to optimal are those that:

- maintain a low tree height,
- do not cause excessive costs in handling individual pages,
- e.g., the leaf degree **r** is not too small relative to the internal node degree **d**.

Final Conclusions

The implemented B+ tree meets all project requirements:

- supports the full set of CRUD operations,
- automatically maintains a balanced structure,
- enables reuse of deallocated space,
- provides detailed metrics for disk operations.

The conducted experiments confirm that the structure's behavior is consistent with theoretical analysis and demonstrate the impact of implementation parameters on operational performance. The B+ tree proves to be an effective and flexible method for indexing files with dynamically changing content.